Java 8 to Java 9: List Creation Recap

A practical reference for choosing mutable lists, fixed-size views, read-only views, and independent snapshots. Includes Java 10 List.copyOf() as a useful follow-on.

1. At a glance

Approach	Can add/remove?	Can set?	Source changes visible?
new ArrayList<>()	Yes	Yes	Not applicable for a fresh list
Arrays.asList(array)	No	Yes	Yes, array changes are reflected
Collections.unmodifiableList(list)	No	No	Yes, live view
new ArrayList<>(list)	Yes	Yes	No structural changes reflected
unmodifiableList(new ArrayList<>(list))	No	No	No structural changes reflected
List.of(elements) [Java 9]	No	No	No source-array changes reflected
List.copyOf(list) [Java 10]	No	No	No structural changes reflected

2. Pros, cons, and when to use each

Approach	Benefit	Caveat	Best use case
ArrayList	Flexible mutable list	Exposing it permits caller changes	Shopping cart, editable working data
Arrays.asList	Concise fixed-size array view	set() works; array and view share storage	Interoperate with an array
unmodifiableList(list)	Read-only access to live list	Original owner can still change it	Service exposes current state safely
new ArrayList<>(list)	Independent mutable container	Shallow copy; extra allocation	Caller needs its own editable list
unmodifiableList(new ArrayList<>(list))	Unmodifiable structural snapshot	Verbose; shallow copy; allows null	Java 8 snapshot API
List.of(...)	Concise unmodifiable list	Rejects null; shallow immutability	Fixed supported options/constants
List.copyOf(list)	Concise unmodifiable snapshot	Rejects null; may reuse immutable input	Snapshot an existing collection

3. Key memory models

```
Same reference:
  original ----> [ArrayList A] <---- alias

Shallow copy:
  original ----> [ArrayList A] --+--> [Payment object]
  copy --------> [ArrayList B] --+

Unmodifiable live view:
```

```
readOnly ----> [Wrapper] ----> [Original ArrayList]
```

Same reference: both variables access one list. **Shallow copy:** two list containers, but element objects may be shared. **Live view:** a wrapper blocks mutations through that wrapper but reads the original list.

4. Code examples

```
String[] array = {"Java", "Python"};
List<String> fixed = Arrays.asList(array);
fixed.set(0, "Go");     // allowed; array[0] is now "Go"
// fixed.add("Rust");  // UnsupportedOperationException
```

```
List<String> original = new ArrayList<>(List.of("UPI", "CARD"));
List<String> live = Collections.unmodifiableList(original);
List<String> snapshot = Collections.unmodifiableList(
    new ArrayList<>(original));
original.add("CASH");
System.out.println(live);     // [UPI, CARD, CASH]
System.out.println(snapshot); // [UPI, CARD]
```

```
String[] languages = {"Java", "Python"};
List<String> stable = List.of(languages);
languages[0] = "Go";
System.out.println(stable); // [Java, Python]
// stable.set(0, "Go");  // UnsupportedOperationException
```

5. Rules that prevent common mistakes

final is not immutable: final List<String> x = new ArrayList<>() prevents reassigning x, not x.add(...).

Unmodifiable is not deeply immutable: List.of(payment) prevents replacing the Payment reference in the list; it does not prevent payment.method = "CARD".

Arrays.asList is fixed-size: set() works, add()/remove() do not. When supplied an existing array, the list is backed by that array.

Snapshot is about list membership: shallow snapshots preserve element references, not necessarily mutable element state. For transaction history, use immutable transaction objects or defensive copies of elements as needed.

Nulls: ArrayList and Arrays.asList allow null; List.of and List.copyOf reject null. Unmodifiable wrappers inherit the contents of their backing list.

6. Decision guide

Need edits? Use ArrayList. **Need a read-only live view?** Use Collections.unmodifiableList(original). **Need a read-only snapshot from an existing list?** Use List.copyOf(original) on Java 10+, or an unmodifiable defensive copy on Java 8. **Know the fixed elements upfront?** Use List.of(...) on Java 9+. **Need array-backed fixed-size behavior?** Use Arrays.asList(array).

7. Quick knowledge check

```
List<String> original = new ArrayList<>();
original.add("Java");
List<String> view = Collections.unmodifiableList(original);
List<String> copy = List.copyOf(original);
original.add("Python");
// view  = [Java, Python]
// copy  = [Java]
```